

XML External Entity (XXE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

XML permite definir 'entidades' (variables de texto) en un DOCTYPE, incluidas entidades que cargan contenido EXTERNO (un fichero local o una URL). Si la app procesa XML que tu controlas y el parser no desactiva esas entidades externas, defines una que apunte a /etc/passwd y el parser te devuelve su contenido.

Muchos parsers XML traen la resolución de entidades externas activada POR DEFECTO por compatibilidad, y el programador no la desactiva explícitamente. El parser confía en que el XML que procesa es de confianza, pero si TU controlas el XML de entrada, controlas sus entidades.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Solo aplica cuando la app procesa XML que tu controlas: subida de ficheros XML/SVG/DOCX/XLSX, SOAP, SAML, o cualquier import que acepte XML. No aplica si la app solo usa JSON.

Cómo identificarla

- La app acepta subida o entrada de XML, SVG, SOAP, SAML o documentos Office (DOCX/XLSX usan XML internamente).
- El fichero XML que subes se procesa (parsea) en vez de guardarse tal cual.
- Errores de parser XML visibles en la respuesta ante un XML malformado.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sube un XML con una entidad que apunte a un fichero que sepas que existe (file:///etc/passwd) y mira si el contenido vuelve reflejado.

Evidencia esperada: El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml
```

Objetivo y contexto: Construye el XML mínimo: declara una entidad externa 'xxe' que apunta a un fichero local, y la referencia dentro del documento.

Resultado esperado: El fichero xxe.xml listo para enviar. El contenido de /etc/passwd debería sustituir a &xxe; si el parser resuelve la entidad.

Capa 2: prueba documentada

```
curl -sS -X POST $URL/upload --data-binary @xxe.xml -H 'Content-Type: application/xml'
```

Objetivo y contexto: Envía el XML malicioso al endpoint que lo procesa. El Content-Type application/xml asegura que el servidor lo trate como XML y no como texto plano.

Resultado esperado: Si el parser es vulnerable, la respuesta contiene las líneas de /etc/passwd (root:x:0:0...) en vez de procesar el documento normalmente.

Anatomía de los payloads

La primera prueba es `printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma que el punto de entrada acepta y PROCESA XML (no solo lo almacena).
- Prueba una entidad local inocua (file:///etc/passwd en tu lab) para ver si se refleja el contenido.
- Si no hay reflexión, monta un XXE ciego: la entidad apunta a tu servidor HTTP/DNS y confirmas por el callback que llega.
- Con lectura de ficheros confirmada, busca configs con credenciales o claves SSH del servidor.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app solo acepta JSON o no procesa XML en ningún punto de entrada.
- El parser tiene DTD/entidades externas desactivadas (comportamiento por defecto en muchas librerías modernas parcheadas).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un parser XML mal configurado resuelve entidades externas que tú defines, dejándote leer ficheros del servidor o hacer peticiones salientes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (xxe)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.